A Reference Architecture for Governance of Cloud Native Applications

William Pourmajidi, Lei Zhang, John Steinbacher, Tony Erwin, and Andriy Miranskyy, *Member, IEEE*

***Abstract*—The evolution of cloud computing has given rise to Cloud Native Applications (CNAs), presenting new challenges in governance, particularly when faced with strict compliance requirements. This work explores the unique characteristics of CNAs and their impact on governance. We introduce a comprehensive reference architecture designed to streamline governance across CNAs, along with a sample implementation, offering insights for both single and multi-cloud environments. Our architecture seamlessly integrates governance within the CNA framework, adhering to a "battery-included" philosophy. Tailored for both expansive and compact CNA deployments across various industries, this design enables cloud practitioners to prioritize product development by alleviating the complexities associated with governance. In addition, it provides a building block for academic exploration of generic CNA frameworks, highlighting their relevance in the evolving cloud computing landscape.**

***Index Terms*—Cloud computing, cloud computing governance, cloud-native applications, cloud native application (CNA) governance, cloud native application reference architecture.**

I. Introduction

Cloud computing has become a primary platform for businesses of all sizes. As predicted by the International Data Corporation FutureScape, 80% of enterprises will put a mechanism in place to shift to cloud-centric infrastructure and applications twice as fast as before the pandemic [1]. cloud's key features, such as scalability, elasticity, ubiquitous access, and pay-as-you-go, coupled with reduced operational costs, render cloud computing appealing to a wide array of organizations [2], [3].

The National Institute of Standards and Technology (NIST) interprets cloud governance as a standardization focus, akin to Internet governance, aiming to ensure interoperability and data portability across various cloud platforms [4]. This perspective anticipates future standards for efficient data and service access tailored to CSCs' needs [5]. Red Hat describes cloud governance as the framework of policies guiding an organization's cloud operations [6]. AWS defines cloud governance as a system allowing customers to set and monitor their cloud requirements in terms of security, cost, and oversight [7]. Azure views governance as a tool for safe cloud adoption, balancing control, security, and compliance with cloud scalability [8]. Google Cloud specializes in its governance definition of data management, ensuring data security, privacy, and compliance with internal and external standards [9]. IBM considers governance as practices to balance benefits, risk, and resources in cloud environments [10], [11]. In this work, **governance** encompasses the management of all aspects of cloud computing and applications, including CNAs, covering functions like monitoring, observability, and compliance for comprehensive cloud management.

A. Cloud Computing Evolution

The past two decades have seen numerous enhancements in cloud services, underpinned by shifts in practices among CSPs and CSCs. These enhancements are largely attributable to changes in cloud infrastructure and cloud application architecture [12].

1) Cloud Infrastructure: The underlying cloud computing infrastructure and technologies surrounding it are constantly changing. The so-called software-defined cloud computing is becoming increasingly popular, and CSPs use advanced virtualization techniques to convert all data center resources to virtualized ones. For example, software-defined networks are replacing traditional networks, and software-defined storage solutions are replacing traditional storage options. On a macro-level, CSPs are evolving their infrastructure management approaches to achieve software-defined data centers [13].

2) Cloud Application Architecture: Cloud application architectures have transformed from monolithic to dynamic, microservice-based designs, meeting cloud environments' scalability and flexibility needs [14]. This shift necessitates lightweight and efficient computing, making containers the preferred choice for their microservice compatibility [15]. Consequently, Kubernetes has become essential for managing and orchestrating microservices [16]. Moreover, the rise of Serverless computing offers an event-driven model, enhancing efficiency by utilizing resources only as needed, in contrast to traditional Virtual Machines (VMs), which often require continuous resource allocation regardless of usage, impacting financial feasibility and resource utilization [17].

Received 3 January 2025; revised 29 April 2025; accepted 6 June 2025. Date of publication 10 June 2025; date of current version 8 September 2025. This work was supported in part by the Natural Sciences and Engineering Research Council of Canada under Grant # RGPIN-2022-03886 and in part by the IBM Centre for Advanced Studies (Project #1142). Recommended for acceptance by K. Djemame. *(Corresponding author: William Pourmajidi.)*

William Pourmajidi and Andriy Miranskyy are with Toronto Metropolitan University, Toronto, ON M5B 2K3, Canada (e-mail: william.pourmajidi@torontomu.ca; avm@torontomu.ca).

Lei Zhang is with the University of Maryland, College Park, MD 20742 USA (e-mail: leizhang@umbc.edu).

John Steinbacher is with IBM Canada Lab, Toronto, ON L6G 1C7, Canada (e-mail: jstein@ca.ibm.com).

Tony Erwin is with IBM Cloud Platform, Austin, TX 78758 USA (e-mail: aerwin@us.ibm.com).

This article has supplementary downloadable material available at https://doi.org/10.1109/TCC.2025.3578557, provided by the authors.

Digital Object Identifier 10.1109/TCC.2025.3578557

2168-7161 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:43:16 UTC from IEEE Xplore. Restrictions apply.

B. The Rise of CNAs

CNAs have emerged as a modern class of cloud applications, inherently designed to leverage the unique capabilities of cloud environments. The Cloud Native Computing Foundation (CNCF) defines CNAs as software systems specifically built to exploit cloud innovations, emphasizing their potential to deliver resiliency and scalability when designed accordingly [18]. Given their increasing importance, CNAs have become a critical focus area in cloud computing. However, the lack of comprehensive governance architectures presents significant risks in their design and implementation [19], [20], [21]. This paper introduces a reference architecture to address these governance challenges. For a more detailed discussion of CNA characteristics, please refer to Section II-A.

C. CNA Governance

CNA Governance, drawing inspiration from cloud and IT governance, involves strategic, architectural, operational, and compliance elements within CNAs [22]. It aligns CNAs' use with business goals and regulatory requirements, incorporating necessary policies and guidelines for resource management. Governance also encompasses security for threat protection, compliance for legal adherence, and performance management for meeting Service Level Agreements (SLAs). Particularly in regulated industries, a CNA cannot operate unless its adherence to all aspects of compliance requirements is met, making governance an indispensable aspect.

CNAs bring complex governance challenges due to their microservice architectures and deployment in orchestrated environments like Kubernetes clusters [16]. They offer agility and scalability, key attributes augmented by serverless computing. While serverless computing efficiently optimizes resource usage, it also introduces additional complexities in governing ephemeral resources [17]. Another layer of complexity in CNA governance stems from the need to implement distinct scalability strategies for both stateless and stateful components.

Moreover, the continuous evolution of CNAs is driven by methodologies such as Continuous Integration and Continuous Delivery (CI/CD) and practices like Everything as Code (XaC) and Infrastructure as Code (IaC). Tools like Terraform [23], Chef [24], and Ansible [25] are integral to these methodologies, facilitating consistent and automated deployment. This ever-changing landscape of CNAs underscores the need for an adaptable and continuous governance framework. Our proposed reference architecture is designed to meet these dynamic requirements, providing a comprehensive solution to the governance challenges of CNAs. For readers seeking a more in-depth exploration of CNA governance challenges, please refer to Section II-B for further details.

D. Our Contribution

This paper navigates the complexities of governing CNAs by developing a targeted reference architecture for CNA governance. The proposed reference architecture and its reference implementation on AWS and IBM public clouds are designed to simplify governance processes in CNAs, particularly critical for regulated industries. By embedding essential governance aspects like monitoring, observability, and compliance into the architecture, we significantly reduce the complexity for cloud practitioners when integrating these elements into their CNAs. This integration fosters a comprehensive and efficient governance model tailored to the unique demands of CNAs.

Our investigation is guided by three Research Questions (RQs) as follows.

- *RQ1:* What distinct characteristics define CNAs?
- *RQ2:* What are the desired characteristics of a reference architecture for CNA governance?
- *RQ3:* How can the software architecture guide the creation of a custom reference architecture for governing CNAs, integrating RQ1 characteristics and RQ2 components?

To address these research questions, we employed a methodology grounded in software engineering principles, specifically through the lens of functional and non-functional requirements analysis. Our approach involved systematically analyzing the challenges presented by each RQ and iteratively designing a governance-oriented reference architecture. This methodology enabled us to balance theoretical insights with practical design considerations while ensuring that the resulting architecture remains extensible, implementation-agnostic, and suitable for multi-cloud environments.

The remainder of this paper is organized as follows. In Section II, we dive into the characteristics of CNAs, addressing RQ1 by elaborating on what precisely defines these applications. This section also explores the challenging landscape of CNA governance, offering insights into our first research question. We continue in Section III by discussing the desired characteristics of a reference architecture for the governance of CNAs, thus answering RQ2. This exploration leads to Section IV, where we present our proposed reference architecture in detail, answering RQ3 and using the insights from the preceding sections. To demonstrate practical applicability, Section V provides real-world examples of how this architecture could be instantiated. A review of related literature, covering CNAs and their governance challenges, is presented in Section VI. We bring the paper to a close in Section VII, summarizing our findings and contributions.

II. Background and Motivation

This section serves as the foundation for our exploration, delving into the defining characteristics of CNAs and the inherent governance challenges they present. In Section II-A, we identify key aspects that distinguish CNAs, addressing our RQ1 and laying the groundwork for understanding their governance complexities. Following this, Section II-B examines the challenges in governing these dynamic and scalable applications, while Section II-C explores broader cloud governance aspects. Together, these insights form the motivation for our work, explaining why a tailored governance approach for CNAs is essential.

A. Characteristics of CNAs

CNAs are exclusively designed, developed, deployed, and maintained for cloud environments, fully utilizing cloud-specific resources and services. Contrary to common assumptions,

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:43:16 UTC from IEEE Xplore. Restrictions apply.

merely running applications on virtualized cloud infrastructure does not qualify them as CNAs. Virtualization and cloud infrastructure are essential but not exhaustive components of CNAs. Distinguishing between generic cloud-based applications and true CNAs is vital for implementing effective governance solutions tailored to the nature of the deployed application [26]. In this section, we outline and discuss the defining characteristics of CNAs to aid practitioners in this differentiation.

1) Microservice Architecture: The defining attribute of CNAs lies in their microservice architecture [27], [28]. This architectural style involves decomposing large, complex applications into atomic, isolated services. These services, each responsible for a specific piece of business logic, communicate with each other predominantly through lightweight, often RESTful, APIs [29]. This encourages a modular approach, where components can be developed, deployed, and scaled independently. Such decentralization not only increases agility but is also crucial for scaling applications efficiently within CNAs [30]. The microservice architecture offers significant benefits in terms of flexibility and scalability, aligning with the dynamic nature of cloud environments and the continuous integration and delivery models they often employ.

2) Containers as Computing Platform: In line with the microservice architecture of CNAs, containers emerge as a fitting computing platform. Given that each microservice component is responsible for atomic business logic and requires limited computational power, containers offer a more appropriate solution than VMs [31], [32]. This suitability stems from containers' ability to be easily managed, replicated, and scaled, in contrast to VMs, which might provide more resources than necessary for microservices. The shared core operating system kernel in containers not only facilitates efficient hardware use but also aligns with the principles of cloud computing that prioritize optimal resource utilization and efficiency. Recognizing this, CSPs have advanced their services, including AWS Elastic Kubernetes Service [33], Azure Kubernetes Services [34], Google cloud Kubernetes Engine [35], and IBM Cloud Kubernetes Service [10], to provide CSCs with robust options for managing and orchestrating containers, thereby streamlining the implementation of microservices in CNAs.

3) Serverless as Computing Platform: CNAs are composed of various components with different run-time requirements. For instance, while the user-interface components and Online Transaction Processing services need to be up and running almost $24/7$, batch processing tasks, aggregations, and backup operations demand a shorter and less frequent run-time. Thus, dedicating a VM or container to a service that runs once or a few times a day is an infeasible under-utilization [36]. The better utilization of resources on the CSPs translates to a more feasible solution for the CSCs [37]. CSPs have brought up a new computing platform known as serverless to address schedulable workloads. Serverless computing hides server usage from developers and runs the associated code on-demand, only charging the user based on the time that code is running [38]. Interestingly, serverless computing and microservices have a lot of common characteristics. If a microservice does not need to be up and running at all times, a serverless computing platform can bring even more financial feasibility than containers [39], [40]. CSPs have developed and released services such as AWS Lambda [41], Azure Serverless [42], Google cloud Serverless Computing [43], and IBM Cloud Code Engine [44].

4) State Isolation: CNAs are built and implemented with a clear separation between stateless and stateful services. In the context of CNAs, even if stateless services interact with stateful services, this interaction is often managed via a set of exposed APIs [45]. This separation allows different scalability practices for each stateless and stateful component. Stateless components, often implemented as containers or serverless functions, require less effort for scalability and reliability. In contrast, stateful component scalability and reliability are not as straightforward and require architectural consideration, including replication and storing the state at any given time. An isolated state is one of the desired characteristics of the CNAs [27], [46].

B. CNA Governance Challenges

In addressing CNA governance challenges, we encounter complexities across architectural components and data types. This includes microservice distribution, container dynamics, serverless platform limitations, diverse telemetry and their Big Data trail, and infrastructure governance intricacies. Each element poses unique governance hurdles, and we briefly explore these to understand effective CNA governance.

1) Microservice Challenges: Microservice architecture imposes a few different types of governance challenges. Given their distributed nature, each microservice is deployed on an isolated computing platform such as a container. Often, there is a high degree of dependency among microservices, and failure in one may cause a ripple effect, and the entire system may be impacted. Hence, in addition to the internal governance of the microservices and their computing platform, one needs to consider the complex relationships among them to construct a representative view of the entire deployed CNA [47].

2) Container Challenges: Containers are among the essential building blocks of CNAs. Microservices are often deployed on a wide range of containers that may or may not be part of the same cluster. Collecting and analyzing telemetry from widely distributed containers is already a challenging task [48]. As the number and nature of the containers running a particular CNA are dynamic, the governance solution must be aware of any potential changes to the infrastructure [49]. That is why many practitioners use governance tools such as Kubernetes on top of container orchestration solutions [50].

3) Serverless Challenges: CSPs that offer Function-as-a-Service (FaaS), also known as serverless platforms, often provide generic dashboards to monitor the current state of processes running on FaaS platforms. However, these dashboards offer limited features such as visualizing the workload, reports on resource utilization, and run-time logs. As the root access to the underlying operating system is not provided in a FaaS offering, installation and configuration of any additional governance tool on them is not possible [51].

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:43:16 UTC from IEEE Xplore. Restrictions apply.

4) Diverse CNA's Telemetry: The array of computing platforms and cloud services in CNAs results in a wide range of telemetry data, including logs, metrics, and traces. Processing this heterogeneous data for meaningful analysis involves cleaning, formatting, and aligning it using time as a unifying element. This cross-referencing is crucial for effective observability in CNAs, facilitating the management of their operational health. Addressing the complexity of diverse telemetry data is essential, as highlighted in several studies, underscoring the need for methodical and standardized data management [52], [53], [54], [55].

5) Big Data Characteristics of CNA's Telemetry: The extensive scale of cloud components in CNAs generates a vast volume of data, exemplifying the 5Vs of Big Data—Volume, Velocity, Variety, Veracity, and Value. This governance data, featuring high volume and velocity, is derived from a wide variety of sources, each contributing to its complexity. The veracity of this data is vital, as it must be accurate and reliable for effective decision-making. The value derived from this telemetry data is pivotal for enhancing the governance and operational efficiency of CNAs. This substantial data stream, marked by its Big Data attributes, necessitates robust solutions for storage, processing, and analysis to effectively manage and utilize this wealth of information for governance purposes. The challenges in handling such Big Data aspects of cloud telemetry have been explored in existing research, highlighting the need for sophisticated and scalable solutions tailored to the unique requirements of CNAs [56], [57], [58], [59], [60], [61], [62], [63].

6) Infrastructure Governance Challenges: Deploying CNAs effectively on Infrastructure as a Service (IaaS) and Platform as a Service (PaaS) demands thorough infrastructure governance, which is central to comprehensive cloud governance. In these environments, practitioners must incorporate governance solutions, key for monitoring and managing CNA deployments. This governance aspect entails synthesizing diverse telemetry data from numerous sources to create an accurate representation of the CNA's operational state. The complexity of managing CNA infrastructure, given the variety of data sources and the depth of information, is critical. Previous research has explored these challenges, underlining the evolving demands of infrastructure monitoring in the context of CNAs [57], [60], [61].

C. CNA Governance Aspects

In this section, we discuss key aspects of CNA governance, highlighting the unique challenges of monitoring, observability, and compliance. These crucial components span a broad spectrum of technical, financial, and legal considerations, forming the foundation of our comprehensive approach to CNA governance. We will explore each area to unravel the complex nature of managing and maintaining CNAs effectively.

1) Monitoring Challenges: In the dynamic field of cloud computing and cloud governance, CSPs are tasked with ensuring the reliability, scalability, and security of their platforms. Concurrently, CSCs face the challenge of rigorous monitoring to uphold SLAs. This dual responsibility has given rise to sophisticated monitoring needs intertwined with the concept of observability. Observability, in this context, is the capability to infer the status of complex systems through their output data. It is a critical component in the governance of CNAs [64], [65], [66], [67], [68], [69].

2) Observability Challenges: Observability, a concept foundational to cloud computing governance, plays a vital role in the effective visualization and prediction of system behaviors. It encompasses the collection and analysis of system data, predominantly focusing on logs, metrics, and traces. Crucial to observability is the balance between data comprehensiveness and maintaining optimal system performance. This often involves implementing adaptive sampling techniques, ensuring that data collection is as efficient as it is informative, and facilitating deeper insights into CNA operations [70], [71], [72].

3) Compliance Challenges: In the cloud environment, particularly for CNAs operating in regulated industries, compliance is an essential concern. Solutions deployed must adhere to specific functional and non-functional requirements, often intertwined with SLAs. Observability, therefore, emerges as a key element in ensuring adherence to these stringent requirements. By leveraging observability tools, organizations can continuously monitor and verify their compliance with relevant standards and regulations, ensuring that their cloud solutions remain within the bounds of mandated guidelines [73]. Compliance often involves demonstrating that the system meets specific controls outlined in established frameworks such as NIST, FEDRAMP, HIPAA, SOC2, and C5. These frameworks enumerate hundreds of controls across categories like access management, data protection, and incident response. For example, adherence to HIPAA mandates safeguards for patient data, while SOC2 emphasizes security, availability, and confidentiality controls. Observability plays a critical role in continuously verifying that these controls are met, providing organizations with the evidence required to demonstrate compliance under audit and maintain adherence to SLAs. CSPs often employ observability tools to collect evidence and ensure compliance with these frameworks. For instance, IBM utilizes its Security and Compliance Center (SCC) and internal tools like "fetcher" and "checker" to automate evidence collection and verification. These tools exemplify how CSPs can enhance compliance by integrating observability with automated compliance checks, enabling organizations to continuously monitor and demonstrate that required controls are implemented and adhered to effectively.

III. Desired Characteristics of a Reference Architecture for CNA Governance

This section addresses RQ2 by outlining the desired characteristics of a reference architecture tailored to CNA governance, leveraging insights from our extensive experience and empirical research. It includes non-functional requirements in Section III-A, architectural requirements in Section III-B, and delves into the construction, deployment, and maintenance requirements in Section III-C. Furthermore, key components crucial for effective governance are detailed in Section III-D. This

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:43:16 UTC from IEEE Xplore. Restrictions apply.

structured approach ensures the architecture supports CNAs optimally, emphasizing performance, reliability, and compliance across various governance facets.

A. Desired Non-Functional Requirements

For the effective governance of CNAs, a reference architecture must incorporate certain non-functional requirements. Drawing on our extensive experience in CNAs design and implementation, and based on a set of non-functional requirements in software engineering [74], [75], [76], we identify the following set of non-functional requirements as foundational for ensuring CNAs' optimal operation. This list provides a solid foundation but can be adjusted by practitioners to include or exclude additional non-functional requirements as deemed appropriate, ensuring alignment with the specific governance needs and unique characteristics of CNAs.

1) Performance and Reliability: A desired aspect of the reference architecture is incorporating software-defined load-balancers to evenly distribute workload among clusters. This approach is fundamental to achieving the desired levels of performance and reliability, ensuring CNAs function seamlessly under various operational demands.

2) Adaptability: The reference architecture should be inherently adaptable to a range of cloud platforms and CNA configurations. This flexibility allows for modifications to the architecture to address the unique requirements of different deployment environments.

3) Extensibility: Extensibility is crucial for a reference architecture to accommodate evolving governance needs. The architecture must be designed with the capacity to be extended and customized, enabling practitioners to tailor it to their specific operational requirements.

4) Elasticity: Auto-scaling and Intelligent Workload Management (IWM) are essential for efficient resource utilization. The reference architecture should advocate for elasticity, utilizing auto-scaling and IWM solutions to enforce optimized workload strategies [77] from prominent CSPs such as AWS Auto Scaling [78], Azure Autoscale [79], Google cloud Autoscaling [80], and IBM Cloud Autoscaling [81] to ensure resources are optimally managed.

5) Integrability: The ability to seamlessly integrate with existing or future platforms is a desired characteristic [82]. The reference architecture needs to ensure components are sufficiently flexible and replaceable, facilitating easy integration and thus enhancing the governance framework's integrability.

6) High Availability: In cloud and distributed systems, ensuring high availability is critical and stands as a fundamental non-functional requirement [83]. A reference architecture must integrate strategies for high availability, including the use of load balancers, clusters, and especially an auxiliary data bus to maintain continuous communication among components, even in the event of primary channel disruptions. Such a design, emphasizing failovers and redundancy upfront, is essential for achieving uninterrupted governance processes and resilience [84]. This approach underscores the pivotal role of high availability in supporting the robustness and reliability of CNAs.

B. Desired Architectural Requirements

For the effective governance of CNAs, the architectural foundation of a reference architecture is critical, including a series of architectural characteristics essential for ensuring optimal flexibility and reliability. Based on our experience, we outline these architectural requirements, which are essential for building a framework that adapts to the changing nature of CNAs.

1) Layered Architecture: A layered architecture is a key characteristic desired for effective CNA governance, enabling scalability and controlled management of microservice-based data flows. This approach aligns with the non-functional requirements detailed in Section III-A, emphasizing the architecture's adaptability to various complexities of CNA deployments. The significance of employing a layered architecture in CNA governance is further explored in Section IV-C, highlighting its crucial role in ensuring flexible and efficient data management throughout the system [85], [86].

2) Adapting Well-Architected Framework Principles for Governance: Drawing on the foundational principles of the AWS Well-Architected Framework [87] and the IBM Cloud for Financial Services [88], it is desired for a reference architecture to integrate these comprehensive guidelines to ensure secure, high-performing, and efficient governance of CNAs. The AWS framework, with its emphasis on operational excellence, security, reliability, performance efficiency, cost optimization, and sustainability [89], offers a versatile set of core concepts applicable across diverse cloud environments.

Similarly, the IBM Cloud for Financial Services framework focuses on risk management and cloud adoption for regulated industries, incorporating robust security and compliance features [90], [91]. A desired architectural requirement is to adapt and extend these frameworks' principles to a broader governance scope, providing a holistic management perspective for CNAs. Practitioners looking to implement or enhance their governance strategies are advised to explore these frameworks for in-depth guidance.

C. Desired Construction, Deployment, and Maintenance Requirements

For effective governance across the lifecycle of CNAs, a comprehensive reference architecture must fulfill specific construction, deployment, and maintenance requirements. These requirements, drawn from extensive field experience and inspired by existing cloud-native architectures [92], [93], [94], [95], ensure that the architecture supports a wide range of governance activities effectively. They address the essential aspects of adaptability, resilience, and efficient management crucial for CNAs operating in dynamic cloud environments.

1) Vendor Agnostic: Essential for broad applicability, the architecture should be designed to be vendor-agnostic, and capable of implementation across all CSPs without dependency on specific proprietary technologies or platforms.

2) Support for Multi-Cloud: Embracing multi-cloud environments is a desired requirement for enhancing the availability and resilience of CNAs. Recognizing the value of deploying across various CSPs, this architecture aims to support a

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:43:16 UTC from IEEE Xplore. Restrictions apply.

multi-cloud approach, facilitating a broader range of deployment possibilities and redundancy strategies [96]. Such flexibility is crucial for tailoring governance mechanisms to leverage the strengths of each CSP effectively. Architectural considerations for data replication and consistency across platforms are essential in multi-cloud setups, ensuring unified governance despite the distributed nature of the resources. This desired feature acknowledges the need for strategic planning in the replication of governance-related data storage, promoting seamless operation across diverse cloud environments.

3) Data Source and Data Type Agnostic: A crucial desired feature of a governance framework is its ability to operate independently of the specific data sources or types it manages. This agnosticism towards data source and type is fundamental for ensuring the framework's adaptability across various computing environments. Embracing widely recognized data management patterns, such as Publish/Subscribe (Pub/Sub) [97], and Message Queue (MQ) [98], enhances this adaptability [99]. These methodologies facilitate efficient data retrieval and dissemination, regardless of the underlying infrastructure, making the architecture versatile and broadly applicable within CNAs.

4) XaC and IaC: In an effective governance framework, XaC and IaC are necessary for managing system components through code, improving traceability and automation across cloud computing. IaC, as part of XaC, plays a key role in CNAs management by facilitating seamless delivery and deployment and ensuring high levels of automation [100], [101], [102]. These approaches offer key benefits such as consistency, scalability, version control, auditability, and portability, critical for various code-as-policy implementations, including IaC, a crucial feature for CNAs [103], [104]. This framework leverages XaC and IaC principles, utilizing platforms like GitHub for source code management [105] and tools like Terraform for resource orchestration [23] to achieve a comprehensive and automated governance model.

5) Gitops, Devops, Devsecops: GitOps, an operational framework evolved from DevOps practices, has become instrumental in managing CNAs effectively. It extends DevOps and DevSecOps best practices to include XaC, including configurations, infrastructure settings, policies, and deployment constraints. This approach combines IaC with DevOps principles, leading to a fully automated solution delivery model known as GitOps. Central to GitOps is the use of version control systems, like GitHub [105], as the single source of truth, enabling traceable and reversible states for deployment using CI/CD [106], [107]. CNAs, often built upon microservices and deployed over containers, leverage the latest in CI/CD advancements within the DevOps paradigm [27], [45], [100]. This setup allows for each microservice to have its own deployment pipeline, enabling simultaneous updates across different CNA components. Adhering to GitOps principles defined by Weaveworks, the entire system of a CNA is described declaratively, with the desired state versioned and changes approved through pull requests or merge requests. Moreover, GitOps' emphasis on observability ensures any divergence from the system's desired state is promptly detected and corrected [107], [108]. This comprehensive integration of GitOps and CI/CD practices makes it an apt framework for the development and deployment of CNAs, aligning with current continuous deployment heuristics [109].

6) End-to-End Continuous Governance: For effective governance of CNAs across their lifecycle, it is essential to embed a comprehensive, end-to-end governance framework that seamlessly integrates with the Software Development Life Cycle (SDLC). The proposed framework consists of four stages to ensure comprehensive governance.

- *Code-level governance:* This initial stage mandates the continuous review of source code against predefined standards and benchmarks with each update or change request, ensuring code quality and compliance from the outset.
- *CI-level governance:* At this juncture, the focus shifts to verifying the integration of various components within CI pipelines, aligning them with overarching governance goals, and ensuring that the integrated components meet all necessary standards.
- *Pre-CD governance:* Situated at the pivotal intersection of CI and CD, this stage rigorously evaluates component readiness for deployment in diverse environments. It functions as an essential checkpoint, potentially involving a governance team's manual approval to underscore the criticality of meticulous examination prior to automation activation. Successful clearance at this stage triggers the CD pipeline, leading to the deployment of components.
- *Post-CD governance:* Post-deployment, the governance framework transitions into a maintenance mode, where it actively monitors for any deviations from the predefined 'ideal' state, leveraging tools like GitHub as the source of truth. Detected discrepancies trigger corrective actions to realign the current state with the desired one, ensuring continuous compliance and governance integrity.

This structured approach to end-to-end continuous governance, underscored by the principle of continuous oversight and adaptation, is designed to navigate CNAs through governance challenges, promoting a unified and strategic governance methodology [108]. Many compliance frameworks mandate automated test triggering mechanisms that not only execute predefined tests but also record their results for posterity, ensuring traceability and accountability in compliance verification. This further reinforces the importance of continuous governance in maintaining adherence to regulatory standards.

7) Heterogeneous Computing Environments: Embracing diverse operational demands within CNAs requires the desired adaptability to various computing environments [110]. The architecture should ideally support VMs for microservices with continuous operational needs and high resource demands, containers for their scalability and flexibility in microservice architecture, and serverless computing for its efficiency in event-driven and intermittent tasks. Given the continuous advancements in technology, such as Kubernetes' exploration into enhanced GPU management [111], it is crucial for the architecture to adapt and provide governance solutions that are specifically tailored to each computing environment. This approach is aimed at ensuring effective governance across all platforms, thereby enhancing the operational efficiency and security of CNAs within the diverse landscape of cloud computing.

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:43:16 UTC from IEEE Xplore. Restrictions apply.

8) High Availability Heartbeat for Reliable Governance: Ensuring uninterrupted governance in CNAs requires a high availability heartbeat mechanism as a key desired feature. This system continuously monitors the operational health of the governance components, acting as an essential tool for maintaining governance reliability. By identifying potential system failures early and activating necessary failover procedures, the heartbeat mechanism ensures governance operations remain continuous, even in the face of disruptions. This capability is particularly critical in the dynamic and often unpredictable cloud environment, reinforcing the architecture's resilience and adherence to governance standards.

D. Desired Components for Governance of CNAs

To achieve comprehensive governance of CNAs, it is necessary to identify desired components within a reference architecture that are crucial for maintaining CNAs' integrity, security, and operational efficiency. Recognizing that some components are inherently composite, combining multiple sub-components to cover various aspects of governance, this discussion provides an in-depth examination of each to clarify their individual and combined contributions. By integrating these desired components, the framework aims to foster a robust and effective governance system, ensuring a resilient and adaptive governance structure tailored to the unique needs of CNAs. **The list of desired components are**:

1) Data ingestion,
2) Fault-tolerant data bus,
3) Data manipulation,
4) Data storage (mutable and immutable), and
5) Data analytics.

The details are given below.

1) Data Ingestion: For the comprehensive governance of CNAs, the inclusion of a *Data ingestion* component is essential. This component, ideally composite in nature, should consist of *Data collector agents* and an *API gateway* to address the diverse computing environments of CNAs effectively. *Data collector agents* are crucial for fetching and pulling data from varied sources, adapting to heterogeneous computing environments like VMs, containers, and serverless platforms, employing patterns such as the sidecar in VM environments or node-based and pod-based methods in containerized setups. In serverless contexts, where system-level access might be limited, reliance on CSP-provided telemetry or code-level instrumentation becomes necessary for gathering application-specific telemetry. Simultaneously, an *API gateway* enhances data ingestion through its dual capability of accepting pushed data and fetching or pulling data, thereby offering a flexible and versatile approach to data collection. This duality is especially critical in CNA environments where data sources are numerous and widely distributed, necessitating adaptable data ingestion strategies. Major CSPs, including AWS [112], Microsoft Azure [113], Google Cloud Platform (GCP) [114], and IBM Cloud [115], support this two-way functionality, which is integral for the governance framework, enabling effective monitoring, observability, and compliance. Through the integration of these desired components, the architecture seeks to ensure the thorough and adaptable ingestion of data, a foundation for effective CNA governance.

2) Fault Tolerant Data Bus: In the governance of CNAs, a *Fault tolerant data bus* is a desired component, essential for managing large volumes of cloud-generated telemetry and facilitating efficient inter-service communication [61]. This component, incorporating both a primary and an auxiliary data bus, ensures resilience and continuous operation, which is especially crucial for monitoring, observability, and compliance within CNAs. The primary data bus, typically employing Pub/Sub or MQ models, is supported by major CSPs, including AWS Simple Queue Service [116], Azure Service Bus [117], Google Cloud Pub/Sub [118], and IBM Cloud Message Queue [119]. In the event of service disruptions, the auxiliary data bus offers a robust alternative, ensuring that governance processes remain uninterrupted. Designed to operate seamlessly across VMs, containers, and serverless platforms, this dual data bus system employs peer-to-peer networks for enhanced flexibility and resilience, which is critical for the dynamic environments of CNAs. This setup underscores the importance of fault tolerance in maintaining a consistent and reliable governance framework. Gossip-based peer-to-peer networks [120] or more specialized versions for content distribution [121] can be employed, enhancing flexibility and resilience across different cloud scenarios. This strategic approach ensures that governance processes remain continuous and uninterrupted, even in the face of primary data transfer service disruptions. For simplicity, in the rest of this paper, we will refer to these two types of data buses as *Data bus*.

3) Data Manipulation: In the context of effective CNA governance, incorporating *Data manipulation* as a desired composite component in a reference architecture is crucial. This component, encompassing *Converter, Filter, Aggregator, and Archiver* sub-components, is pivotal for refining and manipulating raw telemetry data for insightful governance analysis. The *Converter* ensures data uniformity by transforming diverse telemetry into a standardized format, addressing the challenge posed by the variety of formats. This uniformity is essential for simplifying governance processes and enhancing data analysis efficacy. The *Filter's* role is to isolate only the most relevant data for analytics, ensuring efficiency and relevance in data processing. Subsequently, the *Aggregator* organizes this filtered data into manageable subsets, facilitating a structured analysis. Finally, the *Archiver* is tasked with securely storing these subsets in both immutable and mutable storage options outlined in the *Data storage* component. This careful approach to data preservation ensures that governance requirements are consistently met, emphasizing the importance of data integrity and accessibility in the governance framework.

4) Data Storage: For the effective governance of CNAs, *Data storage* is a desired component. This component, composite in nature, consists of *Immutable data storage* and *Mutable data storage*. *Immutable data storage* plays a critical role in maintaining the integrity and authenticity of digital evidence, which is crucial for compliance and legal frameworks. It is designed to ensure that logs and system-generated telemetry,

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:43:16 UTC from IEEE Xplore. Restrictions apply.

once written, cannot be altered or deleted. This immutability is highlighted in previous research, pointing to its significance in preserving the admissibility of digital evidence [61], [122], [123]. Although blockchain technology presents a robust, tamper-proof option for such storage needs [124], its cost-effectiveness for handling extensive data volumes can be challenging [125], [126]. Alternatively, CSPs have developed Write Once Read Many (WORM) storage solutions, effectively preserving data authenticity without the high costs associated with blockchain [127], [128], [129], [130], [131].

Conversely, *Mutable data storage* addresses a broader spectrum of storage needs within cloud computing, offering flexibility in terms of data access, modification, and deletion. This variety is crucial for governance, allowing for the dynamic management of data that aligns with operational requirements and governance standards. CSPs offer an array of software-defined storage solutions, catering to diverse needs in data type, size, security, and retention policies [132]. For everyday storage tasks, services like AWS S3 [133] are suitable [134], while AWS Glacier provides a cost-effective solution for long-term data archiving needs [135]. This dual approach to data storage, encompassing both immutable and mutable solutions, is a desired characteristic within the proposed reference architecture, ensuring comprehensive data management that supports the integrity, security, and compliance of CNAs.

While this discussion focuses on fundamental storage options in both mutable and immutable forms, modern storage architectures also include advanced solutions such as data warehouses, data lakes, and data marts. These options offer increased scalability, flexibility, and accessibility, making them suitable for addressing diverse analytical and operational requirements.

5) Data Analytics: Incorporating pluggable *Data analytics* as a desired component within a reference architecture is crucial for ensuring effective governance of CNAs. These components, specifically tailored to address different governance aspects such as monitoring, observability, and compliance, play a significant role in the architecture. Each data analytics component, designed to be atomic and specialized, concentrates on distinct governance requirements, ensuring a seamless fit within the overarching governance framework. For instance, hypothetical examples of data analytics components could include anomaly detection, drift detection, SLA breach detection, Compliance breach detection, and synthetic monitoring, among others. We recommend practitioners develop and plug in their own data analytics components as necessary to cater to their specific governance needs and objectives.

The diversity in the types of assessments conducted by these components—explicit, range, and baseline—demonstrates their comprehensive approach to data analysis. Explicit assessments ensure compliance with predefined criteria, such as verifying the correct number of nodes in a Kubernetes cluster. Range assessments monitor metrics to stay within established limits, like maintaining acceptable database response times. The most complex baseline assessments develop a model of the 'desired' state to identify and address deviations, underlining the necessity for adaptive governance [59], [60], [63]. These analytics components are indispensable for recognizing deviations from governance standards, including SLAs and configurations, by analyzing data to ascertain if a resource aligns with its defined ideal state. Detected discrepancies prompt immediate actions, enabling proactive governance. Our prior research attests to the efficacy of these data analytics in analyzing cloud-generated data in real-time, underscoring their indispensable role in a governance framework. This adaptability and specialization ensure that the architecture can meet the intricate requirements of cloud governance effectively, highlighting the importance of data analytics as a desired component for comprehensive CNA governance.

IV. Proposed Reference Architecture

We present our proposed reference architecture in Section IV-A. In Section IV-B, we align the desired governance components (discussed in Section III-D) with the proposed reference architecture. Then, in Section IV-C, we provide a detailed overview of the data flow within the reference architecture and among its components. Section IV-D presents our reference architecture using an architectural pattern template [136]. We provide a comparative analysis of our proposed reference architecture in Section IV-E. Finally, in Section IV-F, we discuss the integration of existing tools with the proposed reference architecture to enhance governance practices.

A. Description

CNAs embody the advanced capabilities of cloud technology. Utilizing microservice architectures, APIs, and CI/CD pipelines, CNAs operate within diverse computing environments like containers, serverless frameworks, and VMs. These features render CNAs ideal for enterprise-scale software development. Non-functional requirements such as performance, reliability, and stability are critical, particularly in meeting Quality of Service standards as per SLAs. This is even more pertinent in regulated industries where compliance audits are routine.

The reference architecture we propose integrates governance at its core, offering cloud practitioners, especially in regulated sectors, a blueprint for CNA design and implementation. This integration of governance into the architecture's foundation streamlines the inclusion of governance elements, sparing practitioners the effort of integrating external systems. This architecture, developed by experienced cloud professionals and researchers, is designed to handle the complexities of large-scale system monitoring. In particular, a version of this architecture is actively used within IBM Cloud's observability practices, where it leverages artificial intelligence for near real-time anomaly detection [60].

1) Core Characteristics: The reference architecture is shown in Fig. 1. Our proposed architecture consists of three unique features to address the challenges listed in Section II-B as follows.

- *Multi-vendor cloud support:* The architecture is tailored for multi-vendor cloud environments, enhancing fault tolerance and availability. Fig. 1 illustrates this with two CSPs, `CSP A` and `CSP B`, demonstrating the architecture's flexibility and reliability across various cloud services.

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:43:16 UTC from IEEE Xplore. Restrictions apply.

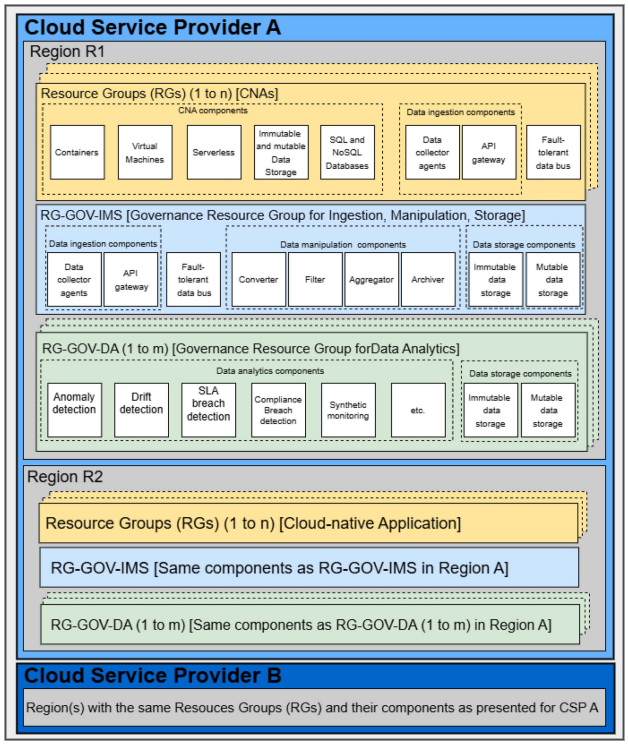


Fig. 1. Graphical representation of the reference architecture. The CNA and Data Analytics components presented here serve as examples. Practitioners may select components that are most applicable to their specific CNA requirements and data analytics needs.

- *Multi-region deployment:* To address geo-redundancy and provide location-aware services, the architecture incorporates multi-region deployment within each CSP. Fig. 1 illustrates this with two regions, Region `R1` and Region `R2`, though the actual number of regions can vary based on service availability and CSP presence in each region.
- *End-to-end governance:* The architecture integrates governance across XaC, IaC, and DevOps processes, leveraging GitOps principles to ensure comprehensive governance coverage. From the initial source code in the Git repository to the final deployment, every stage is designed to meet governance standards, ensuring the process and final solution are fully compliant with governance requirements.

2) Resource Group Deployment: Within each CSP and within each region, multiple resource groups (RGs) are implemented as a means to allocate and manage collections of cloud resources at once. Being a common tool for cloud resource management, RGs are supported by all major CSPs [137], [138], [139], [140].

- *RGs for CNAs:* Each CNA is placed in an isolated RG with all its operational components. Within RGs 1 to n, we deploy one CNA per RG (i.e., we will have n CNAs in total). For governance practices within CNAs, the architecture separates data ingestion, manipulation, and storage components from data analytics[1]. This separation allows multiple teams to run their own data analytics on their managed RGs, accessing the same data without duplicating it.
- *Centeralized RG for GOVernance data Ingestion, Manipulation, and Storage (RG-GOV-IMS):* `RG-GOV-IMS` serves as the centralized hub for consolidating the IMS of telemetry data from all CNAs across RGs 1 to n. This setup ensures that all governance-related data are centrally located, simplifying replication processes, and providing an efficient means for various data analytics components to access the same data. This structure promotes effective governance practices by facilitating the management and analysis of essential governance data.
- *Dedicated RGs for GOVernance Data Analytics (RG-GOV-DA):* `RG-GOV-DA` serves as a dedicated RG for all types of data analysis against data collected in `RG-GOV-IMS`. Its modular structure allows practitioners to create a dedicated RG for different teams to accommodate their specific governance analytics needs.

[1]Data analytics components are vital for analyzing data in real-time or near-real-time, providing actionable governance insights. Refer to Section III-D for more details.

B. Aligning Governance Components With the Proposed Architecture

Now that we have explained the flow of data between the internal components of the proposed reference architecture, we can explore the mapping between these components and the desired components for the governance of CNAs, discussed in Section III-D.

The first composite component *Data ingestion* is strategically placed in two distinct locations in our reference architecture: within RGs 1 to n and in `RG-GOV-IMS`. The *Data ingestion* components located within RGs 1 to n are primarily responsible for data collection from the CNAs and their various components. These components are fundamental for gathering telemetry data from various elements of the CNA, ensuring that all relevant information is efficiently captured and relayed. On the other hand, the *Data ingestion* component located in `RG-GOV-IMS` is tasked with ingesting data from the *Fault tolerant data bus* in RGs 1 to n into `RG-GOV-IMS`). Their role is central to the data collection that forms the backbone of our governance framework. This setup ensures that data collected from the CNAs is processed and stored, facilitating comprehensive analysis and decision-making. This dual placement of the *Data ingestion* component optimizes data flow within our architecture by enabling data collection and ingestion at both the source (CNAs) and the destination (IMS). This strategic positioning of the *Data ingestion* components aligns with our overall objective of building an integrated, efficient, and responsive governance system for CNAs.

The second component in our governance architecture is the *Fault-tolerant data bus*. This component is deployed within RGs 1 to n and in `RG-GOV-IMS`. The inclusion of these data buses is crucial for ensuring efficient data flow and governance throughout the entire cloud environment. They serve as critical conduits for transmitting telemetry data and governance-related information. While our architecture diagram distinguishes the data buses in RGs and `RG-GOV-IMS` for clarity, the practical

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:43:16 UTC from IEEE Xplore. Restrictions apply.

implementation may involve utilizing a singular data bus service, should the CSP offer such a unified solution. For instance, services like AWS Kinesis [141] or IBM Event Streams [142] could be employed to streamline the data flow processes. This unified approach not only simplifies the data management but also enhances the governance framework's efficiency by reducing the complexity and potential points of failure in the data transmission paths.

The third desired component for robust governance is *Data storage*, essential for storing and preserving the authenticity of evidential data. Recognizing its vital role in governance, our proposed reference architecture incorporates *Data storage* within both `RG-GOV-IMS` and `RG-GOV-DA`. This setup ensures that all crucial data remains secure and unaltered, a vital aspect for maintaining governance standards. Moreover, mutable storage offers data storage solutions for data whose authenticity does not need to be proven. These components are implemented in both `RG-GOV-IMS` and `RG-GOV-DA`, utilizing either identical or distinct services provided by CSPs based on specific requirements. This component encompasses a range of storage options, from basic object stores like AWS S3 or IBM COS [143] to more complex NoSQL databases such as AWS DynamoDB [144] or IBM Cloudant [145], catering to the varied data storage demands within a governance framework.

The fourth desired component of our governance-focused architecture is the *Data manipulation*, positioned within `RG-GOV-IMS` in our reference architecture. This placement optimally situates this composite component close to both the *Fault tolerant data bus* and *Data storage* components, allowing more efficient transmission and processing of data crucial for governance.

The fifth desired component of our reference architecture is the *Data analytics*. This composite component consists of many data analytics instances, and they are more than just data analysis tools; rather, they encompass specific business logic aimed at addressing diverse aspects of cloud governance, including monitoring, observability, and compliance. Each *Data analytics* component is designed to be focused and atomic, targeting particular governance needs and integrating smoothly into the overall governance framework. To host these *Data analytics* components, a dedicated RG, `RG-GOV-DA` is established. This allocation highlights their importance in identifying deviations from established governance standards and criteria. The data analytics components are essential for analyzing incoming data, ensuring that resources align with their defined desired states, and triggering alerts for proactive governance actions. This design element strengthens the reference architecture's capability to manage the complex and diverse demands of cloud governance.

C. Data Flow

Fig. 2 depicts the internal communication among components of the proposed reference architecture.

1) Data Bus: The reference architecture integrates both a primary and auxiliary data bus in *Fault tolerant data bus* to strengthen governance robustness. This dual data bus system

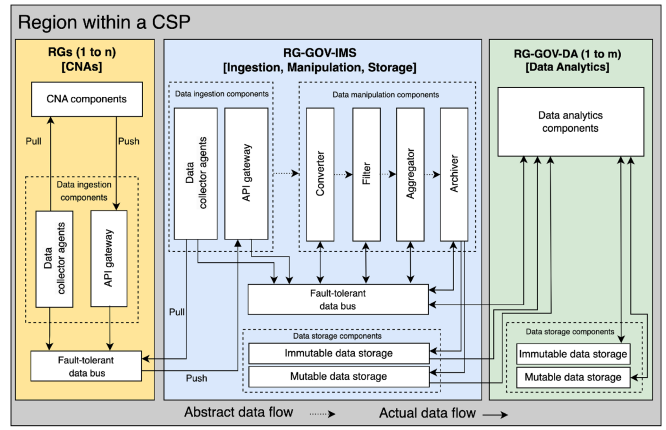


Fig. 2. Internal communication among components of the proposed reference architecture. Fig. 1 shows examples of analytics components in `RG-GOV-DA`. For the sake of brevity, these components are depicted as *Data Analytics Components* in this figure.

ensures uninterrupted data flow, a crucial aspect of comprehensive governance. If the primary data bus encounters issues, the auxiliary data bus temporarily takes over, maintaining data transmission until the primary bus is restored (details in Section III-D). This dual-bus approach significantly enhances governance resilience, ensuring continuous data accessibility for governance-related analysis and decision-making.

Additionally, it is important to consider the payload limitations of data buses in a governance context. In scenarios where large objects need to be transmitted but exceed the data bus's payload capacity, a practical solution is to store the payload in a database or cloud storage. The data bus can then exchange pointers to these stored payloads, efficiently handling large object transmission within the governance framework without overloading the data bus capacity [86]. This method ensures that even large-scale data requirements are met within the governance architecture, maintaining the integrity and efficiency of the governance process.

2) Data Flow Across RGs: In line with Section IV-A2, the architecture utilizes n RGs for hosting CNAs. Within each `RG`, *Data ingestion* components are responsible for gathering telemetry data from CNA components. These components employ either `pull` method [146] by *Data collector agents* or `push` method [146] by *API gateway* to collect telemetry from various components of the CNA. Subsequently, *Data ingestion* submits its collected telemetry to the *Fault tolerant data bus* component within the same `RG`. Given the diverse computing environments employed by CNAs, namely VMs, containers, and serverless platforms, it is essential to deploy *Data collector agents* tailored to each environment. This strategic deployment is crucial for effective governance, ensuring that relevant data from various computing platforms are efficiently captured and processed.

3) Data Flow in `RG-GOV-IMS`*:* In `RG-GOV-IMS`, which is dedicated to data ingestion, manipulation, and storage, the telemetry data from the *Fault tolerant data bus* of n RGs are relayed to either the *Data collector agents* or the *API gateway* components, using either `pull` or `push` methods respectively.

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:43:16 UTC from IEEE Xplore. Restrictions apply.

The choice of method is based on the specific design of each data bus within the RGs.

Once in RG-GOV-IMS, telemetry data are passed through *Data manipulation* sub-components: *Converter*, *Filter*, *Aggregator*, and *Archiver*. These components collectively process the raw telemetry data to make it suitable for governance analysis. The *Converter* standardizes the raw data into a unified format, the *Filter* isolates data pertinent to data analytics components, the *Aggregator* organizes the data into subsets for consumption by data analytics components (see [85], [86] for details), and the *Archiver* stores necessary subsets of data into *Data storage* composite component. This composite component consists of both *Immutable data storage* and *Mutable data storage*, with submissions governed by retention policies that reinforce the architecture's ability to serve CNA governance effectively. The interaction between the *Data ingestion* and the *Data manipulation* components within RG-GOV-IMS is facilitated via its *Fault tolerant data bus* component.

4) Data Flow in RG-GOV-DA*:* Within each RG-GOV-DA, one or more *Data analytics* components are deployed[2]. Each *Data analytics* instance conducts analysis from a distinct governance perspective, converting telemetry data into actionable insights. These insights are then stored within the same RG-GOV-DA, either in immutable or mutable storage options available in *Data storage* based on compliance requirements for data storage.

Note that RG-GOV-IMS serves as the primary data source for all *Data analytics* in RG-GOV-DA 1 to m. These *Data analytics* components access real-time data via the *Fault tolerant data bus* and historical data from *Data storage* components within RG-GOV-IMS, enabling them to perform comprehensive governance analytics.

D. Architectural Pattern

Finally, we summarize information about our reference architecture using an architectural pattern template [136, Sec. 13.1] as follows.

1) Context: CNAs represent an advanced stage in the evolution of application development. These applications, while demonstrating agility and scalability, require robust governance frameworks. Such frameworks extend beyond mere monitoring and observability to include compliance, security, and performance management. Ensuring that CNAs operate reliably and align with both regulatory requirements and organizational goals is essential for their successful deployment and management.

2) Problem Description: CNAs' core characteristics, like heterogeneous computing platforms and microservice architecture, present significant governance challenges. Each computing platform demands specific methods for log collection and analysis. It is crucial to have a detailed view of each component and a holistic view of the entire platform to accurately depict the system's current state. These challenges include managing the distributed nature of microservices, limitations in serverless platforms, and the dynamic aspects of containers. There is also a need to handle diverse telemetry data efficiently, ensuring clean, formatted, and cross-referenced data to reflect an accurate system state. Moreover, governance in CNAs must account for Big Data characteristics inherent in telemetry, demanding advanced solutions for effective storage and analysis. These complexities, detailed in Section II-B, underscore the need for a sophisticated and comprehensive governance framework that can address the unique hurdles presented by each architectural element and data type within CNAs.

3) Solution Description: This reference architecture is tailored for cloud governance, specifically addressing the complexities of CNA governance. It serves as a toolkit for cloud practitioners engaged in designing and implementing CNAs across both regulated and non-regulated industries. Adopting a "batteries-included" approach, the architecture provides the necessary components to ensure effective governance within cloud environments. Our extensive and ongoing collaboration with IBM has empirically validated this framework, reinforcing its practical applicability.

a) Advantages: The architecture, having been applied in real-world CNA scenarios, effectively addresses governance needs. It features a layered, modular design, allowing practitioners to adapt and customize components to fit their unique governance requirements. Being cloud- and technology-neutral, it offers versatility across various cloud services. Based on the latest innovations in cloud computing, as detailed in Section I-A, the architecture supports the creation of new governance frameworks or the enhancement of existing ones (see Section IV-F for details).

b) Disadvantages: While CNAs commonly utilize cloud technologies, their architectures can vary, often shaped by specific governance needs. Therefore, adopters of this architecture must judiciously assess its suitability for their specific CNA architecture and selectively incorporate the components that align with their governance objectives. While the architecture provides a blueprint of crucial components for governance, it does not offer detailed implementation guidance for each, placing the onus on practitioners to determine the most suitable technologies for each component in their governance framework.

Additionally, while the modular nature of the architecture allows for flexibility, it may introduce integration complexity, particularly when aligning existing tooling with governance pipelines. In high-throughput CNA environments—where telemetry data may be generated at extreme volumes—certain components such as real-time ingestion or analytics layers may become bottlenecks if not scaled appropriately. Moreover, because the architecture offers a generalized reference model rather than prescriptive implementation steps, adopters must invest effort into aligning it with specific compliance regimes, organizational policies, or performance thresholds. This can result in increased time-to-deployment or additional resourcing for architecture customization and validation.

[2]Multiple data analytics components often generate alerts that necessitate further analysis and aggregation to discern actual issues and minimize false alerts, thereby reducing "alert fatigue" [60]. These components can send alerts to a specific topic on the data bus, which are then evaluated by a correlation analytics component. This component identifies genuine alerts, either through predefined rules or machine learning models, creating a bidirectional flow between the *Data bus* in RG-GOV-IMS and the *Pluggable Data Analytics (DA)* in RG-GOV-DA.

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:43:16 UTC from IEEE Xplore. Restrictions apply.

E. Comparative Analysis of Reference Architectures

Comparative analysis is crucial to evaluate the effectiveness of a reference architecture in meeting established industry standards. We assess the architecture's alignment with key software engineering and architectural criteria that underpin system design and operational excellence in Section IV-E1. In Section IV-E2, we compare our proposed architecture with the comprehensive governance standards. Through these comparisons, we aim to illustrate our architecture's adherence to industry benchmarks for security, reliability, and compliance, ensuring a governance framework that is both rigorous and resilient.

1) Software Engineering and Software Architecture Comparison: In the realm of CNAs, aligning reference architectures with established software engineering and architectural standards is important. These criteria, or quality attributes, serve as fundamental benchmarks that ensure the architecture not only meets functional requirements but also excels in non-functional dimensions such as maintainability, scalability, and security. Our approach in this comparative analysis is to evaluate how our proposed reference architecture stands against these critical quality attributes, drawing insights from "Software Architecture in Practice" by Bass et al. [136].

To articulate the degree of alignment, we categorize our architecture's support into two distinct classifications:

- *Fully-supported:* This indicates that the architecture either directly addresses the requirement or provides the flexibility that allows adopters to integrate their solutions seamlessly.
- *Partially-supported:* This denotes cases where the architecture conforms to as many compliance requirements as feasible but cannot fully address certain needs. In such instances, responsibility for implementation rests with the adopter.

The proposed architecture demonstrates strong alignment with critical software engineering and architecture quality attributes. A summary of this alignment is provided below, while a detailed breakdown is presented in supplementary materials (SM) SM.A, which shows that the architecture fully supports 86% of the evaluated attributes, addressing essential aspects such as availability, performance, and security. The remaining 14% are partially supported, requiring some degree of customization or external integration.

2) Cloud Control Matrix Comparison: The Cloud Control Matrix (CCM), developed by the Cloud Security Alliance (CSA), is a recognized framework for cloud computing security, offering a detailed guide across 17 domains to ensure comprehensive governance, from risk management to incident response. This framework is essential for organizations utilizing CNAs, serving as a benchmark to assess the security and compliance of cloud services. It has been applied in both academia and industry to evaluate various aspects of cloud governance [147], [148], [149]. The proposed architecture aligns with the CCM across multiple governance domains. A summary of this alignment is provided below, with a detailed evaluation presented in SM.B, which reveals that 65% of the evaluated domains are fully supported, covering critical areas such as audit, encryption, and governance. Additionally, 24% are partially supported, requiring some degree of customization, while 12% are out of scope.

F. Integrating and Advancing Existing Solutions for Enhanced Cloud Governance

Incorporating established tools such as Elasticsearch [150], Logstash [151], Kibana [152], Mezmo [153], Splunk [154], and Sysdig [155] within our governance framework is a strategic approach, leveraging their unique capabilities to enhance functionality. Our architecture emphasizes leveraging these existing resources, steering clear of the complexities involved in developing components from scratch. By channeling telemetry data to *Data collector agents* in `RG-GOV-IMS`, we facilitate seamless integration of telemetry and alerts into a unified RG. To ensure consistency across diverse telemetry formats, we employ *Data manipulation* sub-components for standardization.

Many governance tools provide standardized mechanisms to either push data (e.g., through webhooks or streaming APIs) or allow periodic data pulls (e.g., via scheduled queries or REST APIs). These mechanisms are typically backed by persistent storage layers, including relational databases, NoSQL engines, or other storage systems. The proposed architecture can accommodate these data exchange strategies by aligning them with the ingestion pathways defined in the *Data collector agents* and *Data manipulation* stages within the `RG-GOV-IMS` resource group. These integration points make it feasible to import telemetry, audit trails, or configuration data from a variety of third-party tools, providing both observability and compliance enrichment while avoiding redundant tooling or duplicated pipelines.

Ultimately, adopters of our reference architecture can efficiently instantiate additional instances using IaC, reducing the time and effort required compared to building an original implementation. This approach enhances the governance architecture's adaptability and scalability and ensures a more resource-efficient deployment.

V. Real-World Instantiation of the Proposed Reference Architecture

We implemented the proposed reference architecture, emphasizing its adaptability and scalability across multi-cloud environments. The deployment demonstrates the functionality of resource groups in handling telemetry and log data from high-demand and less-demanding environments, showcasing key architectural elements. The implementation includes:

A representative subset of the architecture is deployed, with AWS hosting a high-demand CNA environment and IBM Cloud hosting a less-demanding CNA environment. Governance components (`RG-GOV-IMS` and `RG-GOV-DA`) are implemented on AWS for consistency.

CNAs are containerized using Docker and deployed on both AWS and IBM. These CNAs emit telemetry data, including metrics and logs, every second for a total of 4,000 entries. AWS employs additional components, such as API Gateways

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:43:16 UTC from IEEE Xplore. Restrictions apply.

and SQS, to handle the high velocity of data, while IBM utilizes a streamlined approach to transmit telemetry directly to governance pipelines.

The `RG-GOV-IMS` resource group processes telemetry data through a structured pipeline, ensuring fault tolerance, scalability, and compliance. It employs API Gateways, SNS topics, and Lambda functions for ingestion, manipulation, and storage. Immutable and mutable S3 storage options are used for logs and metrics, respectively, with object-lock enabled for regulatory adherence.

The `RG-GOV-DA` resource group analyzes telemetry data for performance evaluation. It measures time delays across pipeline stages, comparing AWS and IBM implementations. Insights include variations in delay times due to inter-regional data transfer and internal architecture differences.

We demonstrate the system's ability to maintain performance consistency across cloud providers. A graphical representation of the data flow within the instantiated reference architecture is available in SM.C., providing a visual walkthrough of the end-to-end operational workflow.

This deployment also serves as a practical case study to guide cloud and cloud-native adopters in implementing governance architectures tailored to their own environments. Recognizing that no two CNA deployments are alike, the proposed framework is intentionally designed for flexibility, allowing organizations to incorporate additional components or modify existing ones to meet their specific needs.

For comprehensive implementation details—including deployment instructions, telemetry formats, data flow, time-delay analytics, and cross-cloud performance metrics—as well as a detailed discussion of architectural design decisions and tradeoffs, please refer to SM.C.

The GitHub repository [156] contains example code, deployment scripts, and documentation structured around GitOps principles, facilitating modular, reproducible multi-cloud deployments.

VI. Related Literature

This section explores the literature related to CNAs, covering monitoring, observability, compliance, and governance. We discuss CNA monitoring, observability, and compliance in Section VI-A, highlighting the need for integrated approaches in these areas. Section VI-B focuses on CNA governance, while Section VI-C examines existing tools and practices.

Given the interdisciplinary nature of this work and the wide range of adjacent research domains, we have limited our literature review to peer-reviewed studies that focus specifically on CNA-related monitoring, observability, compliance, and governance. This targeted scope ensures relevance to the core components of our proposed architecture and avoids diluting the discussion with broader cloud or software engineering topics not directly aligned with CNA governance.

A. CNA Monitoring, Observability, and Compliance

Monitoring CNAs is a complex task due to their deployment over elastic platforms, utilization of various computing environments (VMs, containers, serverless), and microservice architectures. Both detailed and unified views of applications are essential for the effective governance and maintenance of CNAs.

Brunner et al. [157] highlight that CNAs inherently support self-management, scalability, and resilience across clustered units of application logic. However, there is no dominant application development model for CNAs, presenting challenges in establishing comprehensive governance models. This gap is where our work aims to make a significant contribution.

Oliveira et al. [158] stress the importance of operational visibility in cloud services' success. They point out the complexities in monitoring cloud environments and the need for adaptable solutions to the diverse and changing nature of cloud resources. Their work on a unified monitoring and analytics pipeline, OpVis, provides insights into addressing some of these challenges, although focusing primarily on the observability aspect of CNAs.

However, comprehensive governance (including compliance, security, and policy adherence) is not addressed in [157], [158]. Our proposed reference architecture (see Section IV) aims to bridge this gap by providing a holistic governance framework that encompasses monitoring, observability, compliance, and security, thus ensuring robust and efficient management of CNAs.

Torkura et al. [159] shed light on the compliance challenges specific to CNAs. They underscore CNAs' vulnerability due to the diversity of technologies used and rapid development cycles. They highlight the inadequacy of traditional security assessments in dynamic CNAs, where microservices are frequently created and terminated. To address these issues, they propose new approaches like the Security Gateway, dynamic document store, and security health endpoints integrated into the CNA workflow. Torkura et al. indicate that existing research primarily focuses on security mechanisms such as authentication, authorization, and network security, leaving a gap in comprehensive vulnerability detection and security assessments.

Our work aligns with the insights of Torkura et al. recognizing the need for a governance framework that goes beyond monitoring and observability to include compliance. We aim to fill the gaps identified by Torkura et al. embedding compliance within our reference architecture. This approach ensures that CNAs not only meet operational requirements but also comply with security standards and regulatory demands. Our architecture, therefore, provides a comprehensive solution to the complex compliance landscape of CNAs, where existing research is insufficient.

B. CNA Governance

In their study, Laszewski et al. [160] explore the evolution of cloud computing governance. They emphasize that cloud computing offers superior security and governance compared to traditional data center operations. Major cloud vendors continuously integrate new security patterns and compliance requirements into their services, ensuring customers benefit from

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:43:16 UTC from IEEE Xplore. Restrictions apply.

enhanced security and governance. By using cloud services, organizations automatically align with these evolving standards, often without incurring additional costs. However, leveraging cloud security effectively requires meticulous implementation and ongoing governance. CSPs offer a wide range of services to address various security needs, but it is the responsibility of individual CSP to implement these services properly. Maintaining and monitoring the implemented security measures demands substantial governance efforts. Cloud services offer tools for monitoring, auditing, and configuration control, creating an overlay of governance activities. This unified cloud governance model ensures consistency across deployed workloads, avoiding gaps that may arise in on-premises deployments due to complexity or cost. The authors also highlight various governance guidelines designed to support secure and compliant operations in the cloud. These guidelines cater to different industry verticals, data requirements, and external compliance needs, offering pathways to enhance cloud security postures. Even for organizations without specific industry compliance requirements, adopting these governance guidelines can significantly enhance the resilience and security of cloud resources, supporting internal audit and control processes. Laszewski et al. emphasize the importance of these governance models in ensuring strict adherence to security and compliance standards in cloud environments, a critical factor for effective CNA governance.

Kosinska et al. [161] delve into governance for cloud-hosted applications, including CNAs. They compare cloud computing to an advanced form of service-oriented computing (SOC), where cloud platforms offer not just data and applications but also a range of computing resources as services. This characteristic makes cloud-hosted applications act like services, often depending on other cloud applications or interacting with remote clients via APIs. This similarity with SOC suggests that many SOA governance concepts are directly applicable to cloud platforms and cloud-hosted applications. The authors define governance for cloud-hosted applications as ensuring these applications operate as expected while maintaining a certain quality of service. They focus on three governance features: policy enforcement, formulating performance SLOs, and application performance monitoring (APM). Policy enforcement involves ensuring compliance with various standards and practices, such as dependency management and software versioning requirements. Performance SLOs are statistical bounds on application performance used to negotiate SLAs and monitor application consistency. APM involves continuous monitoring of cloud-hosted applications to detect and diagnose performance anomalies. However, current cloud technologies do not satisfactorily implement these governance features, leading to the emergence of third-party governance solutions. These external services, while offering access control and API management, are often expensive and require additional programming or configuration. They also suffer from limited visibility and control over applications residing in the cloud. Kosinska et al. argue for built-in CNA governance capabilities, which they demonstrate as more robust, effective, and user-friendly compared to external solutions. Furthermore, they highlight the rapid proliferation of web-accessible APIs and the need for automated API governance. Modern applications increasingly rely on remote web APIs, reducing programming and maintenance workloads. This growing reliance on APIs calls for new techniques in API governance, focusing on consistent policy implementation across multiple APIs. Kosinska et al.'s research targets API governance in cloud-hosted web applications, proposing systems for configuring and enforcing policies at the API level and stipulating performance SLAs for individual APIs. This study by Kosinska et al. contributes to our understanding of cloud governance, particularly in the context of cloud-hosted applications and APIs. It underscores the need for integrated CNA governance mechanisms, highlighting the limitations of current approaches and advocating for more robust solutions that are in tune with the unique demands of cloud environments.

C. Existing Tools and Practices in Governance of CNAs

Toka et al. [162] highlight the significance of fault detection in Cloud applications. They propose an open-source, CNA monitoring solution that emphasizes the need for lightweight yet effective tools in dynamic cloud environments. Their solution integrates elements such as collectors, databases, and visualization engines, underscoring the need for comprehensive monitoring frameworks.

Wang et al. [163] introduce CloudRanger, a novel root cause analysis tool designed for CNAs. CloudRanger's ability to adapt to the dynamic nature of cloud infrastructures exemplifies the complexity of monitoring cloud applications. The tool's unique approach to establishing dynamic, causal relationships and constructing impact graphs for applications without a fixed topology is particularly noteworthy.

These monitoring tools and methodologies, while effective, typically focus on isolated aspects of CNAs, such as fault detection or root cause analysis. However, in the broader context of cloud governance, there is a need for an integrated approach that encompasses not only monitoring and observability but also compliance, security, and policy management. Our proposed reference architecture (see Section IV) aims to fill this gap. By integrating existing tools and practices with additional governance components, we provide a holistic framework for managing CNAs. This integrated approach ensures that CNAs are not only monitored for performance and reliability but also governed in line with compliance requirements and security best practices, ultimately enhancing the overall management and operational efficiency of CNAs.

VII. Conclusion

This paper explores the realm of CNAs and the challenges involved in governing them. We present a reference architecture for robust CNA governance that is applicable across a range of single and multi-cloud environments. Rather than proposing a one-size-fits-all model, the architecture is designed to be modular and adaptable, enabling integration with diverse deployment contexts and enterprise governance policies. By embedding governance directly into CNA design, the framework

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:43:16 UTC from IEEE Xplore. Restrictions apply.

offers a "batteries-included" approach that streamlines compliance efforts and enables cloud professionals to focus on core product development. The architecture's real-world instantiation illustrates its flexibility and demonstrates how it may be tailored to meet specific organizational, regulatory, or operational needs. While this work highlights broad applicability, we also recognize that cloud environments vary in scale, tooling, and governance maturity. As such, additional customization may be required in some contexts. Overall, the proposed architecture aims to improve CNA management and operational efficiency, serving as a practical foundation for further research and standardization in cloud-native governance.

Integrating AI and ML into CNA governance frameworks can automate policy enforcement and compliance, transforming regulatory requirements into executable code to enhance consistency and reduce human error. AI-driven predictive analytics can proactively identify compliance risks, enabling timely remediation. Future research should focus on seamlessly incorporating these technologies into governance pipelines to maintain scalability and robustness across diverse cloud environments.

References

[1] "IDC-futurescape," 2020. [Online]. Available: [Online]. Available: http://phc.pt/enews/IDC-FutureScape.pdf

[2] P.-F. Hsu, S. Ray, and Y.-Y. Li-Hsieh, "Examining cloud computing adoption intention, pricing mechanism, and deployment model," *Int. J. Inf. Manage.*, vol. 34, no. 4, pp. 474–488, 2014.

[3] S. V. Nandgaonkar and A. Raut, "A comprehensive study on cloud computing," *Int. J. Comput. Sci. Mobile Comput.*, vol. 3, no. 4, pp. 733–738, 2014.

[4] "NIST cloud computing standards roadmap," Accessed: Aug. 12, 2024. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.500--291r2.pdf

[5] M. Hogan et al., "NIST cloud computing standards roadmap," *NIST Special Publication*, vol. 35, pp. 6–11, 2011.

[6] "What is cloud governance?," Jan. 2023. [Online]. Available: https://www.redhat.com/en/topics/automation/what-is-cloud-governance

[7] AWS, "Aws_cloudgovernance_ebook_driving-success-and-security-in-the-cloud.pdf," 2021. [Online]. Available: https://pages.awscloud.com/rs/112-TZM-766/images/AWS_CloudGovernance_ebook_Driving-Success-and-Security-in-the-Cloud.pdf

[8] "Govern methodology for the cloud - cloud adoption framework| microsoft learn," 2025. [Online]. Available: https://learn.microsoft.com/en-ca/azure/cloud-adoption-framework/govern/methodology

[9] "What is data governance? |Google cloud," 2025. [Online]. Available: https://cloud.google.com/learn/what-is-data-governance

[10] "IBM cloud compliance overview page," 2025. [Online]. Available: https://www.ibm.com/cloud/compliance

[11] "What is data governance? |IBM," 2024. [Online]. Available: https://www.ibm.com/topics/data-governance

[12] B. Varghese and R. Buyya, "Next generation cloud computing: New trends and research directions," *Future Gener. Comput. Syst.*, vol. 79, pp. 849–861, 2018.

[13] A. A. Abbasi et al., "Software-defined cloud computing: A systematic review on latest trends and developments," *IEEE Access*, vol. 7, pp. 93294–93314, 2019.

[14] "What is a container?," 2025. [Online]. Available: https://www.docker.com/resources/what-container

[15] G. Liu et al., "Microservices: Architecture, container, and challenges," in *Proc. IEEE 20th Int. Conf. Softw. Qual. Rel. Secur. Companion*, 2020, pp. 629–635.

[16] "Kubernetes," 2025. [Online]. Available: https://kubernetes.io/

[17] E. Van Eyk et al., "Serverless is more: From paas to present cloud computing," *IEEE Internet Comput.*, vol. 22, no. 5, pp. 8–17, Sep./Oct. 2018.

[18] "Cloud native apps; cloud native glossary," 2025. [Online]. Available: https://glossary.cncf.io/cloud-native-apps/

[19] I. Odun-Ayo et al., "A systematic mapping study of cloud-native application design and engineering," in *Journal of Physics: Conference Series*. Philadelphia, PA, USA: IOP Publishing, 2019.

[20] J. Alonso et al., "Decide: An extended devops framework for multi-cloud applications," in *Proc. 3rd Int. Conf. Cloud Big Data Comput.*, 2019, pp. 43–48.

[21] J. Alonso et al., "Understanding the challenges and novel architectural models of multi-cloud native applications–a systematic literature review," *J. Cloud Comput.*, vol. 12, no. 1, p. 6, 2023.

[22] B. Thuraisingham, "Cloud governance," in *Proc. IEEE 13th Int. Conf. Cloud Comput.*, 2020, pp. 86–90.

[23] "Terraform," HashiCorp, (n.d.),[Online]. Available: https://www.terraform.io/

[24] "Chef software devops automation solutions," Chef, 2025. [Online]. Available: https://www.chef.io/

[25] "Infrastructure automation with ansible," Ansible, 2025. [Online]. Available: https://www.ansible.com/integrations/infrastructure?hsLang=en-us

[26] F. Leymann et al., "Native cloud applications: Why monolithic virtualization is not their foundation," in *Proc. Int. Conf. Cloud Comput. Serv. Sci.*, 2016, pp. 16–40.

[27] N. Kratzke and P.-C. Quint, "Understanding cloud-native applications after 10 years of cloud computing-a systematic mapping study," *J. Syst. Softw.*, vol. 126, pp. 1–16, 2017.

[28] A. Balalaie, A. Heydarnoori, and P. Jamshidi, "Migrating to cloud-native architectures using microservices: An experience report," in *Proc. Eur. Conf. Service-Oriented Cloud Comput.*, 2015, pp. 201–215.

[29] M. Fowler, "Microservices," 2024. [Online]. Available: https://martinfowler.com/articles/microservices.html

[30] H. Zhang et al., "Microservice architecture in reality: An industrial inquiry," in *Proc. 2019 IEEE Int. Conf. Softw. Archit.*, 2019, pp. 51–60.

[31] V. Singh and S. K. Peddoju, "Container-based microservice architecture for cloud applications," in *Proc. Int. Conf. Comput. Commun. Autom.*, 2017, pp. 847–852.

[32] M. Amaral et al., "Performance evaluation of microservices architectures using containers," in *Proc. IEEE 14th Int. Symp. Netw. Comput. Appl.*, 2015, pp. 27–34.

[33] "Managed Kubernetes service," AWS, 2025. [Online]. Available: https://aws.amazon.com/eks/

[34] "Managed Kubernetes service (AKS)," Microsoft, 2024. [Online]. Available: https://azure.microsoft.com/en-us/services/kubernetes-service/#overview

[35] "Kubernetes - google Kubernetes engine (GKE)," Google, 2025. [Online]. Available: https://cloud.google.com/kubernetes-engine

[36] X. Li et al., "The method and tool of cost analysis for cloud computing," in *Proc. 2009 IEEE Int. Conf. Cloud Comput.*, 2009, pp. 93–100.

[37] A. Eivy and J. Weinman, "Be wary of the economics of" serverless" cloud computing," *IEEE Cloud Comput.*, vol. 4, no. 2, pp. 6–12, Mar./Apr. 2017.

[38] P. Castro et al., "The rise of serverless computing," *Commun. ACM*, vol. 62, no. 12, pp. 44–54, 2019.

[39] D. Taibi, J. Spillner, and K. Wawruch, "Serverless computing-where are we now, and where are we heading?," *IEEE Softw.*, vol. 38, no. 1, pp. 25–31, jan./Feb. 2020.

[40] "What are cloud native applications?," VMware, 2025. [Online]. Available: https://tanzu.vmware.com/cloud-native

[41] "Serverless computing - AWS lambda," AWS, 2025. [Online]. Available: https://aws.amazon.com/lambda/

[42] "Azure serverless," Microsoft, 2025. [Online]. Available: https://azure.microsoft.com/en-ca/solutions/serverless/

[43] "Serverless computing," Google, 2025. [Online]. Available: https://cloud.google.com/serverless

[44] "IBM cloud code engine - Canada," IBM, 2025. [Online]. Available: https://www.ibm.com/ca-en/cloud/code-engine

[45] M. Wurster et al., "Cloud-native deploy-ability: An analysis of required features of deployment technologies to deploy arbitrary cloud-native applications," in *Proc. CLOSER*, 2020, pp. 171–180.

[46] C. Fehling et al., *Cloud Computing Patterns: Fundamentals to Design, Build, and Manage Cloud Applications*. Berlin, Germany: Springer, 2014.

[47] J. Levin and T. A. Benson, "ViperProbe: Rethinking microservice observability with eBPF," in *Proc. IEEE 9th Int. Conf. Cloud Netw.*, 2020, pp. 1–8.

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:43:16 UTC from IEEE Xplore. Restrictions apply.

[48] F. Moradi et al., "ConMon: An automated container based network performance monitoring system," in *Proc. 2017 IFIP/IEEE Symp. Integr. Netw. Service Manage.*, 2017, pp. 54–62.

[49] P. Raj et al., "Automated multi-cloud operations and container orchestration," *Softw.-Defined Cloud Centers: Oper. Manage. Technol. Tools*, pp. 185–218, 2018. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-319-78637-7_9

[50] P. Martin, "Observability," in *Kubernetes*. Berlin, Germany: Springer, 2021, pp. 175–183.

[51] R. Cordingly et al., "Enhancing observability of serverless computing with the serverless application analytics framework," in *Proc. Companion ACM/SPEC Int. Conf. Perform. Eng.*, 2021, pp. 161–164.

[52] R. Gatev, "Observability: Logs, metrics, and traces," in *Introducing Distributed Application Runtime (DAPR)*. Berlin, Germany: Springer, 2021, pp. 233–252.

[53] A. V. Miranskyy et al., "An iterative, multi-level, and scalable approach to comparing execution traces," in *Proc. 6th Joint Meeting Eur. Softw. Eng. Conf. ACM SIGSOFT Symp. Foundations Softw. Eng.*, 2007, pp. 537–540.

[54] A. V. Miranskyy et al., "SIFT: A scalable iterative-unfolding technique for filtering execution traces," in *Proc. Conf. Center Adv. Stud. Collaborative Res.: Meeting Minds*, 2008, pp. 274–288.

[55] "Metrics, logs and traces: The golden triangle of observability in monitoring," 2018. [Online]. Available: https://devops.com/metrics-logs-and-traces-the-golden-triangle-of-observability-in-monitoring/

[56] A. Miranskyy et al., "Operational-log analysis for Big Data systems: Challenges and solutions," *IEEE Softw.*, vol. 33, no. 2, pp. 52–59, Mar./Apr. 2016.

[57] W. Pourmajidi et al., "On challenges of cloud monitoring," in *Proc. 27th Annu. Int. Conf. Comput. Sci. Softw. Eng.*, 2017, pp. 259–265.

[58] M. S. Islam and A. Miranskyy, "Anomaly detection in cloud components," in *Proc. IEEE 13th Int. Conf. Cloud Comput.*, 2020, pp. 1–3.

[59] M. S. Islam et al., "Anomaly detection in a large-scale cloud platform," in *Proc. IEEE/ACM 43 rd Int. Conf. Softw. Eng.: Softw. Eng. Pract.*, 2021, pp. 150–159.

[60] W. Pourmajidi et al., "Dogfooding: Using IBM cloud services to monitor IBM cloud infrastructure," in *Proc. 29th Annu. Int. Conf. Comput. Sci. Softw. Eng.*, 2019, pp. 344–353.

[61] W. Pourmajidi et al., "The challenging landscape of cloud monitoring," in *Knowledge Management in the Development of Data-Intensive Systems*. Boca Raton, FL, USA: CRC Press, 2021, pp. 157–189.

[62] S. Sohana et al., "Cloudheatmap: Heatmap-based monitoring for large-scale cloud systems," 2024, *arXiv:2410.21092*.

[63] M. Saiful Islam et al., "Anomaly detection in large-scale cloud systems: An industry case and dataset," 2024, *arXiv:2411.09047*.

[64] "What is cloud native observability?," New Relic, 2024. [Online]. Available: https://newrelic.com/blog/best-practices/what-is-cloud-native-observability#::text=Observability\%20is\%20defined\%20as\%20your,data\%20processes\%2C\%20and\%20hardware\%20processes

[65] C. Esposito, A. Castiglione, and K.-K. R. Choo, "Challenges in delivering software in the cloud as microservices," *IEEE Cloud Comput.*, vol. 3, no. 5, pp. 10–14, Sep./Oct. 2016.

[66] X. Yu et al., "Cloudseer: Workflow monitoring of cloud infrastructures via interleaved logs," *ACM SIGARCH Comput. Archit. News*, vol. 44, no. 2, pp. 489–502, 2016.

[67] "Summary of the Amazon S3 service disruption in the Northern Virginia (US-EAST-1) region," AWS, 2017. [Online]. Available: https://aws.amazon.com/message/41926/

[68] "Amazon and the 150 million typo : The two-way," NPR, 2017. [Online]. Available: https://www.npr.org/sections/thetwo-way/2017/03/03/518322734/amazon-and-the-150-million-typo

[69] A. Hamou-Lhadj, "A call for the development of an international standard for the management of AIOps systems," in *Proc. ASPLOS Workshop Cloud Intell. AIOps*, 2024, Art. no. 1.

[70] M. Bohner and N. Wintz, "Controllability and observability of time-invariant linear dynamic systems," *Mathematica Bohemica*, vol. 137, no. 2, pp. 149–163, 2012.

[71] R. E. Kalman, "On the general theory of control systems," in *Proc. 1st Int. Conf. Autom. Control*, Moscow, USSR, 1960, pp. 481–492.

[72] R. Picoreti et al., "Multilevel observability in cloud orchestration," in *Proc. IEEE 16th Int. Conf. Dependable, Autonomic Secure Comput., 16th Int. Conf. Pervasive Intell. Comput., 4th Int. Conf. Big Data Intell. Comput. Cyber Sci. Technol. Congr.*, 2018, pp. 776–784.

[73] K. P. Joshi, L. Elluri, and A. Nagar, "An integrated knowledge graph to automate cloud data compliance," *IEEE Access*, vol. 8, pp. 148541–148555, 2020.

[74] L. Chung et al., *Non-Functional Requirements in Software Engineering*. Berlin, Germany: Springer, 2012.

[75] S. Paradkar, *Mastering Non-Functional Requirements*. Birmingham, U.K.: Packt Publishing Ltd., 2017.

[76] K. M. Adams et al., *Nonfunctional Requirements in Systems Analysis and Design*. Berlin, Germany: Springer, 2015.

[77] L. Nicoletti et al., "Cross-cloud management of sensitive data via blockchain: A payslip calculation use case," in *Proc. CEUR Workshop*, 2018.

[78] "AWS auto scaling," AWS, 2025. [Online]. Available: https://aws.amazon.com/autoscaling/

[79] "A. autoscale," Microsoft, 2025. [Online]. Available: https://azure.microsoft.com/en-us/features/autoscale/

[80] "Autoscaling groups of instances| compute engine documentation," Google. [Online]. Available: https://cloud.google.com/compute/docs/autoscaler

[81] "Autoscaling," IBM. [Online]. Available: https://cloud.ibm.com/docs/cloud-foundry-public?topic=cloud-foundry-public-autoscale_cloud_foundry_apps

[82] K. Henttonen et al., "Integrability and extensibility evaluation from software architectural models–a case study," *Open Softw. Eng. J.*, vol. 1, no. 1, pp. 1–20, 2007.

[83] M. R. Mesbahi, A. M. Rahmani, and M. Hosseinzadeh, "Reliability and high availability in cloud computing environments: A reference roadmap," *Hum.-Centric Comput. Inf. Sci.*, vol. 8, no. 1, pp. 1–31, 2018.

[84] V. S. Sharma, R. R. Ramnani, and S. Sengupta, "A framework for identifying and analyzing non-functional requirements from text," in*Proc. 4th Int. workshop twin peaks requirements Archit.*, 2014, pp. 1–8.

[85] S. Hoque and A. Miranskyy, "Architecture for analysis of streaming data," in *Proc. IEEE Int. Conf. Cloud Eng.*, 2018, pp. 263–269.

[86] S. Hoque and A. Miranskyy, "Online and offline analysis of streaming data," in *Proc. IEEE Int. Conf. Softw. Architecture Companion*, 2018, pp. 68–71.

[87] "AWS well-architected - build secure, efficient cloud applications," AWS, 2024. [Online]. Available: https://aws.amazon.com/architecture/well-architected/?wa-lens-whitepapers.sort-by=item.additionalFields.sortDate&wa-lens-whitepapers.sort-order=desc

[88] "Reference architecture overview| IBM cloud docs," (n.d.). [Online]. Available: https://cloud.ibm.com/docs/framework-financial-services?topic=framework-financial-services-reference-architecture-overview

[89] "The 6 pillars of the AWS well-architected framework | AWS partner network (APN) blog," 2022. [Online]. Available: https://aws.amazon.com/blogs/apn/the-6-pillars-of-the-aws-well-architected-framework/

[90] "IBM cloud for financial services," IBM, 2025. [Online]. Available: https://www.ibm.com/cloud/financial-services

[91] "Banking industry architecture: Reference diagram - IBM cloud architecture center," IBM, 2025. [Online]. Available: https://www.ibm.com/cloud/architecture/architectures/banking/reference-architecture

[92] "Cloud native architecture," 2025. [Online]. Available: https://architecture.cncf.io/

[93] "Enabling allianz direct's scaling through platform engineering| cloud native architecture," 2024. [Online]. Available: https://architecture.cncf.io/architectures/allianz/

[94] "Scaling adobe's service delivery foundation with a cell-based architecture| cloud native architecture," 2024. [Online]. Available: https://architecture.cncf.io/architectures/adobe/

[95] S. R. Goniwada, *Cloud Native Architecture and Design: A Handbook for Modern Day Architecture and Design With Enterprise-Grade Examples*. Berlin, Germany: Springer, 2022.

[96] R. Moreno-Vozmediano et al., "Orchestrating the deployment of high availability services on multi-zone and multi-cloud scenarios," *J. Grid Comput.*, vol. 16, no. 1, pp. 39–53, 2018.

[97] W. Fang et al., "Design and evaluation of a pub/sub service in the cloud," in *Proc. 2011 Int. Conf. Cloud Service Comput.*, 2011, pp. 32–39.

[98] N.-L. Tran et al., "EQS: An elastic and scalable message queue for the cloud," in *Proc. IEEE 3rd Int. Conf. Cloud Comput. Technol. Sci.*, 2011, pp. 391–398.

[99] "What is pub/sub? The publish/subscribe model explained," 2023. [Online]. Available: https://ably.com/topic/pub-sub

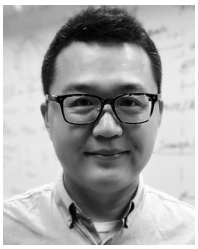
Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:43:16 UTC from IEEE Xplore. Restrictions apply.

[100] "Architecting cloud native. net applications for azure," Microsoft, 2024. [Online]. Available: https://docs.microsoft.com/en-us/dotnet/architecture/cloud-native/
[101] "What is cloud-native? Is it hype or the future of software development?," 2021. [Online]. Available: https://stackify.com/cloud-native/
[102] "Infrastructure as code," Microsoft, 2022. [Online]. Available: https://docs.microsoft.com/en-us/dotnet/architecture/cloud-native/infrastructure-as-code
[103] "Real-time policy enforcement with governance as code - the new stack," 2022. [Online]. Available: https://thenewstack.io/real-time-policy-enforcement-with-governance-as-code/
[104] K. Morris, *Infrastructure as Code: Managing Servers in the Cloud*. Sebastopol, CA, USA: O'Reilly Media, 2016.
[105] "Github: Where the world builds software," GitHub, 2025. [Online]. Available: https://github.com/
[106] "What is gitops?," GitLab, 2025. [Online]. Available: https://about.gitlab.com/topics/gitops/
[107] "Guide to gitops," 2024. [Online]. Available: https://www.weave.works/technologies/gitops/
[108] F. Beetz and S. Harrer, "Gitops: The evolution of devops?," *IEEE Softw.*, vol. 39, no. 4, pp. 70–75, Jul./Aug. 2022.
[109] "Gitops is continuous deployment for cloud native applications," 2025. [Online]. Available: https://www.gitops.tech/
[110] B. Scholl, T. Swanson, and P. Jausovec, *Cloud Native: Using Containers, Functions, and Data to Build Next-Generation Applications*. Sebastopol, CA, USA: O'Reilly Media, 2019.
[111] Schedule GPUs, "Kubernetes", 2022. [Online]. Available: https://kubernetes.io/docs/tasks/manage-gpus/scheduling-gpus/
[112] "API management - amazon API gateway - AWS," 2025. [Online]. Available: https://aws.amazon.com/api-gateway/
[113] "API management–manage API gateways |microsoft azure," 2025. [Online]. Available: https://azure.microsoft.com/en-ca/products/api-management
[114] "API gateway| Google cloud," 2025. [Online]. Available: https://cloud.google.com/api-gateway
[115] "API gateway - IBM API connect," 2025. [Online]. Available: https://www.ibm.com/products/api-connect/api-gateway
[116] "Fully managed message queuing–amazon simple queue service," AWS, 2025. [Online]. Available: https://aws.amazon.com/sqs/
[117] "Azure service bus—cloud messaging service," Microsoft, 2025. [Online]. Available: https://azure.microsoft.com/en-us/services/service-bus/#overview
[118] "Pub/sub for application & data integration," Google, 2025. [Online]. Available: https://cloud.google.com/pubsub
[119] "Introduction to IBM MQ," IBM, 2021. [Online]. Available: https://www.ibm.com/docs/en/ibm-mq/9.0?topic=mq-introduction
[120] K. Birman, "The promise, and limitations, of gossip protocols," *ACM SIGOPS Operating Syst. Rev.*, vol. 41, no. 5, pp. 8–13, 2007.
[121] S. Androutsellis-Theotokis and D. Spinellis, "A survey of peer-to-peer content distribution technologies," *ACM Comput. Surv.*, vol. 36, no. 4, pp. 335–371, 2004.
[122] W. Pourmajidi et al., "Immutable log storage as a service," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng.: Companion*, 2019, pp. 280–281.
[123] D. Reilly, C. Wren, and T. Berry, "Cloud computing: Forensic challenges for law enforcement," in *Proc. Int. Conf. Internet Technol. Secured Trans.*, 2010, pp. 1–7.
[124] W. Pourmajidi and A. Miranskyy, "Logchain: Blockchain-assisted log storage," in *Proc. IEEE 11th Int. Conf. Cloud Comput.*, 2018, pp. 978–982.
[125] A. Chaer et al., "Blockchain for 5G: Opportunities and challenges," in *Proc. IEEE Globecom Workshops*, 2019, pp. 1–6.
[126] W. Pourmajidi et al., "Immutable log storage as a service on private and public blockchains," *IEEE Trans. Serv. Comput.*, vol. 16, no. 1, pp. 356–369, Jan./Feb. 2023.
[127] "IBM 3363 optical worm drive," Computer History Museum, 1987. [Online]. Available: https://www.computerhistory.org/collections/catalog/102671161
[128] "Worm," AWS, (n.d.). [Online]. Available: https://aws.amazon.com/blogs/storage/tag/worm/
[129] "Overview of immutable storage for blob data - azure storage," Microsoft, (n.d.). [Online]. Available: https://docs.microsoft.com/en-us/azure/storage/blobs/immutable-storage-overview
[130] "Protecting cloud storage with worm, key management and more updates," Google, (n.d.). [Online]. Available: https://cloud.google.com/blog/products/storage-data-transfer/protecting-cloud-storage-with-worm-key-management-and-more-updates
[131] "IBM cloud object storage - immutable data," IBM, (n.d.). [Online]. Available: [Online]. Available: https://www.ibm.com/ca-en/cloud/object-storage/immutable
[132] S. Obrutsky, "Cloud storage: Advantages, disadvantages and enterprise solutions for business," in *Proc. Conf.: EIT New Zealand*, 2016.
[133] "Amazon S3 - cloud object storage - AWS," (n.d.). [Online]. Available: https://aws.amazon.com/s3/
[134] "Free cloud computing services - AWS free tier," AWS, 2025. [Online]. Available: https://aws.amazon.com/free/?all-free-tier
[135] "Amazon s3 glacier storage classes," AWS, 2025. [Online]. Available: https://aws.amazon.com/s3/storage-classes/glacier/
[136] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2012.
[137] "What are resource groups? - AWS resource groups and tags," AWS, 2025. [Online]. Available: https://docs.aws.amazon.com/ARG/latest/userguide/resource-groups.html
[138] "Manage resource groups - azure portal - azure resource manager," Microsoft, 2025. [Online]. Available: https://learn.microsoft.com/en-us/azure/azure-resource-manager/management/manage-resource-groups-portal
[139] "Using resource groups — cloud monitoring," Google, 2025. [Online]. Available: https://cloud.google.com/monitoring/groups
[140] "Managing resource groups," IBM, 2025. [Online]. Available: https://cloud.ibm.com/docs/account?topic=account-rgs&interface=ui
[141] "Amazon kinesis - process & analyze streaming data," AWS, 2025. [Online]. Available: https://aws.amazon.com/kinesis/
[142] "Event streams," IBM, 2025. [Online]. Available: https://www.ibm.com/cloud/event-streams
[143] "IBM cloud object storage," 2025. [Online]. Available: https://www.ibm.com/products/cloud-object-storage
[144] "Fast noSQL key-value database–amazon dynamoDB – AWS," 2025. [Online]. Available: https://aws.amazon.com/dynamodb/
[145] "IBM cloudant," [Online]. Available: https://www.ibm.com/products/cloudant
[146] J. Meehan et al., "Data ingestion for the connected world," in *Proc. Conf. Innov. Data Syst. Res.*, 2017, pp. 8–11.
[147] A. Chaudhuri, "Governance and risk management in the cloud with cloud controls matrix V3 and ISO/IEC 38500: 2008," in *Handbook of Research on Security Considerations in Cloud Computing*. Hershey, PA, USA: IGI Global, 2015, pp. 80–101.
[148] A. S. Saidah and N. Abdelbaki, "A new cloud computing governance framework," in *Proc. CLOSER*, 2014, pp. 671–678.
[149] T. Weil, "Risk assessment methods for cloud computing platforms," *IT Professional*, vol. 22, no. 1, pp. 63–66, 2020.
[150] "Elasticsearch: The official distributed search & analytics engine| elastic," 2025. [Online]. Available: https://www.elastic.co/elasticsearch
[151] "Logstash: Collect, parse, transform logs | elastic," 2025. [Online]. Available: https://www.elastic.co/logstash
[152] "Kibana: Explore, visualize, discover data | elastic," 2025. [Online]. Available: https://www.elastic.co/kibana
[153] "Telemetry data pipeline & log analysis solutions| mezmo," 2025. [Online]. Available: https://www.mezmo.com/
[154] "Splunk| the key to enterprise resilience," 2025. [Online]. Available: https://www.splunk.com/
[155] "Sysdig| security for containers, kubernetes, and cloud," 2025. [Online]. Available: https://sysdig.com/
[156] "Williampourmajidi/cna_governance," 2024. [Online]. Available: https://github.com/WilliamPourmajidi/cna_governance
[157] S. Brunner et al., "Experimental evaluation of the cloud-native application design," in *Proc. IEEE/ACM 8th Int. Conf. Utility Cloud Comput.*, 2015, pp. 488–493.
[158] F. Oliveira et al., "A cloud-native monitoring and analytics framework," IBM Research Division Thomas J. Watson Research Center, New York, NY, USA, Tech. Rep. RC25669 (WAT1710-006), 2017.
[159] K. A. Torkura, M. I. Sukmana, and C. Meinel, "Integrating continuous security assessments in microservices and cloud native applications," in *Proc. 10th Int. Conf. Utility Cloud Comput.*, 2017, pp. 171–180.
[160] T. Laszewski et al., *Cloud Native Architectures: Design High-Availability and Cost-Effective Applications for the Cloud*. Birmingham, U.K.: Packt Publishing Ltd, 2018.

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:43:16 UTC from IEEE Xplore. Restrictions apply.

[161] J. Kosińska and K. Zieliński, "Autonomic management framework for cloud-native applications," *J. Grid Comput.*, vol. 18, pp. 779–796, 2020.
[162] L. Toka et al., "Predicting cloud-native application failures based on monitoring data of cloud infrastructure," in *Proc. 2021 IFIP/IEEE Int. Symp. Integr. Netw. Manage.*, 2021, pp. 842–847.
[163] P. Wang et al., "Cloudranger: Root cause identification for cloud native systems," in *Proc. 18th IEEE/ACM Int. Symp. Cluster Cloud Grid Comput.*, 2018, pp. 492–502.

William Pourmajidi received the MSc degree in computer science from Ryerson University, and is a solution and cloud architect with two decades of experience in designing complex information systems. He is currently working toward the PhD degree with the Department of Computer Science, Toronto Metropolitan University (formerly Ryerson University), Toronto, Canada. For his Ph.D., he is focusing on the interplay among software architecture, cloud computing, and cloud governance.



Lei Zhang received the PhD degree in computer science from McMaster University, Hamilton, Canada. He is an assistant professor with the Information Systems Department, University of Maryland, Baltimore County (UMBC), USA. His research interests include software engineering and emerging technologies. He also has more than five years of software development experience as a software engineer and project coordinator.

John Steinbacher received the PhD degree in condensed matter physics and the MBA degree from Queen's University, Kingston, Canada. He is a program director with the Cloud Division, IBM Canada Lab, Markham, Canada. His focus is on building enterprise cloud applications, which include the Console for the IBM Cloud Platform.

Tony Erwin received the MS degree in computer science from Indiana University, Bloomington, IN, USA. He a senior technical staff member with IBM in Austin, TX, USA, where he's an architect in the IBM Infrastructure Artificial Intelligence Center of Excellence (AI COE). Within the AI COE, Tony helps accelerate the adoption of generative AI in internal applications to make employees in the IBM Infrastructure organization happier and more productive. Previously, he was an architect with the IBM Cloud for Financial Services after being the lead architect for the IBM Cloud console. His interests include AI, cloud, security, microservices, and user experience.

Andriy Miranskyy (Member, IEEE) received the PhD degree in applied mathematics from the University of Western Ontario, London, Canada. He is an associate professor with the Department of Computer Science, Toronto Metropolitan University (formerly Ryerson University), Toronto, Canada. His research interests include mitigating risk in software engineering, with a focus on large-scale software systems

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 25,2026 at 15:43:16 UTC from IEEE Xplore. Restrictions apply.